

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE LAB MANUAL



Dr. Muhammad Siddique

Computer Organization & Assembly Language Lab Manual (8086 Assembly using MASM/TASM/emu8086)

Dr. Muhammad Siddique

August 2026

Contents

1	Week 1: Number Systems and Data Representation	2
2	Week 2: 8086 Microprocessor Architecture and Registers	6
3	Week 3: Introduction to the Assembly Programming Environment	10
4	Week 4: Data Transfer Instructions and Addressing Modes	13
5	Week 5: Arithmetic Instructions	17
6	Week 6: Logical and Bit Manipulation Instructions	21
7	Week 7: Control Transfer Instructions – Jumps and Loops	25
8	Week 8: Procedures and the Stack	29
9	Week 9: Arrays and Indexed Addressing	33
10	Week 10: String Manipulation Instructions	38
11	Week 11: Macros and Modular Programming	42
12	Week 12: Interrupts and I/O Programming	46
13	Week 13: Final Project – Simple Calculator and Best Practices	51

1 Week 1: Number Systems and Data Representation

Objective: Understand how a computer represents and manipulates data internally, and become fluent in converting between number systems used throughout the course.

Tasks:

1. **Task 1:** Understand binary, decimal, octal, and hexadecimal number systems, and explain why hexadecimal is the preferred human-readable notation for machine code and memory addresses.
2. **Task 2:** Convert numbers between decimal, binary, octal, and hexadecimal by hand, and verify each conversion using a calculator or emulator's memory/register window.
3. **Task 3:** Perform binary addition and subtraction, and compute 1's and 2's complements, explaining what each complement scheme is used for.
4. **Task 4:** Represent signed integers using sign-magnitude and 2's complement notation, and identify the range of values representable in an 8-bit and a 16-bit register.
5. **Task 5:** Understand ASCII and BCD representation of characters and digits, and explain why DOS interrupt routines expect ASCII-encoded input/output.

Details of Lab Experiment:

Task 1: Number systems used in computer organization

- **Decimal (base 10):** digits 0–9, the system humans use naturally for everyday counting.
- **Binary (base 2):** digits 0 and 1, the native language of digital circuits because a transistor has exactly two stable states (on/off, high/low voltage).
- **Octal (base 8):** digits 0–7, a compact grouping of 3 binary bits; historically popular on machines with word sizes that were multiples of 3 bits.

- **Hexadecimal (base 16):** digits 0–9 and A–F, a compact grouping of 4 binary bits, widely used to represent memory addresses, opcode bytes, and register contents because two hex digits map exactly onto one byte.

Why it matters: every assembler, debugger, and datasheet you will use in this course displays addresses and register values in hexadecimal, so fluency with these conversions is a prerequisite for reading and writing assembly code, not just an academic exercise.

Task 2: Base conversions

Convert $(214)_{10}$ to binary, octal, and hexadecimal.

$$214_{10} = 11010110_2 = 326_8 = D6_{16}$$

Method (decimal to binary): Repeatedly divide by 2 and record the remainders from bottom to top.

$$214 \div 2 = 107 \text{ } r0, \quad 107 \div 2 = 53 \text{ } r1, \quad 53 \div 2 = 26 \text{ } r1, \quad 26 \div 2 = 13 \text{ } r0, \quad \dots$$

Method (binary to hex): Group the binary digits into sets of four starting from the right ($1101\ 0110 \rightarrow D6$), and **binary to octal** groups them into sets of three ($011\ 010\ 110 \rightarrow 326$). Practice this task with at least five different decimal values, including one that produces a non-terminating grouping, so you get comfortable padding with leading zeros.

Task 3: Binary arithmetic and complements

- **1's complement:** invert every bit of the number (change every 0 to 1 and every 1 to 0). It is mainly a stepping stone toward the 2's complement and is rarely used to represent numbers directly because it has two representations of zero (00000000 and 11111111).
- **2's complement:** take the 1's complement and add 1. This is how the 8086 – and virtually every modern processor – represents negative numbers internally, because it allows the same adder circuit to perform both addition and subtraction.

Example: Represent -25 in an 8-bit register using 2's complement.

$$25_{10} = 00011001_2 \quad \Rightarrow \quad 1's \text{ complement} = 11100110_2 \quad \Rightarrow \quad 2's \text{ complement} = 11100111_2$$

Verify this by adding 00011001_2 (25) and 11100111_2 (-25 in 2's complement); the 9-bit result should be 100000000_2 , and discarding the overflow carry out of the 8-bit register leaves exactly 00000000_2 (zero), confirming the representation is correct.

Task 4: Signed integers

In sign-magnitude, the most significant bit indicates sign (0 = positive, 1 = negative) and the remaining bits store the magnitude – simple to understand but awkward for hardware because it needs separate add and subtract circuitry and has two zeros. In 2's complement – the scheme actually used by the 8086 – there is a single representation of zero and arithmetic circuits do not need separate subtraction hardware, since subtraction is implemented as "add the 2's complement."

Range table (memorize this):

- An 8-bit signed register (e.g. AL) can represent values from -128 to $+127$.
- A 16-bit signed register (e.g. AX) can represent values from -32768 to $+32767$.
- An 8-bit unsigned register can represent 0 to 255; a 16-bit unsigned register can represent 0 to 65535.

Understanding these ranges up front explains why the Overflow Flag (OF) exists and why you must be careful when choosing byte-sized versus word-sized operands later in the course.

Task 5: ASCII and BCD

- **ASCII:** a 7/8-bit code representing characters, e.g. 'A' = 41h, '0' = 30h. Note in particular that the ASCII codes for the digits '0'–'9' are 30h–39h; this is why, later in the course, you will repeatedly see code that does `SUB AL, 30H` to turn a typed character into its numeric value, and `ADD AL, 30H` to do the reverse before printing.
- **BCD (Binary Coded Decimal):** each decimal digit is stored in its own 4-bit nibble, e.g. 47_{10} is stored as 0100 0111. BCD trades storage efficiency for the convenience of never needing binary-to-decimal conversion when displaying numbers, which is why some early calculators and point-of-sale systems used it heavily.

Why it matters: the DOS keyboard-input service (INT 21H, function 01H) always returns an ASCII character in AL, never a raw numeric value, so every program in this manual that reads a digit from the user must perform this ASCII-to-numeric conversion before doing arithmetic on it.

Summary:

This week you learned the number systems, conversions, and binary representation schemes (1's/2's complement, ASCII, BCD) that underlie every instruction you will write in assembly language for the rest of the course. Make sure you can convert confidently in both directions before moving on, since every remaining lab assumes this fluency.

2 Week 2: 8086 Microprocessor Architecture and Registers

Objective: Understand the internal architecture of the 8086 microprocessor, its register set, memory segmentation scheme, and the flags register.

Tasks:

1. **Task 1:** Identify the general-purpose registers of the 8086 and their conventional uses, and explain how each 16-bit register splits into two independently addressable 8-bit halves.
2. **Task 2:** Identify the segment registers and understand memory segmentation, and explain why the 8086 needed segmentation given its 20-bit address bus and 16-bit registers.
3. **Task 3:** Identify the pointer and index registers, and describe a typical scenario in which each one is used.
4. **Task 4:** Understand the Flags register and the purpose of each flag, and describe an instruction that would set each flag.
5. **Task 5:** Compute a physical address from a segment:offset pair, and explain why two different segment:offset pairs can point to the same physical byte.

Details of Lab Experiment:

Task 1: General-purpose registers

- **AX (Accumulator):** used for arithmetic, logic, and I/O operations; many instructions (like MUL, DIV) implicitly use AX, so it is the "default" working register.
- **BX (Base):** often used as a base address pointer for indexing into arrays and data structures in memory.
- **CX (Count):** used as a loop/repeat counter; the LOOP and REP-prefixed string instructions automatically decrement CX.
- **DX (Data):** used for I/O port addressing and to hold the high-order 16 bits of 32-bit multiplication/division results together with AX.

Each of these 16-bit registers can also be accessed as two 8-bit halves, e.g. AX = AH (high byte) + AL (low byte). This matters in practice because many DOS service calls only touch AL or AH – for example, function 01H

of INT 21H returns the typed character in AL while leaving the upper byte AH set to the function number, not the character.

Task 2: Segment registers

- **CS (Code Segment):** points to the segment containing the currently executing instructions; combined with IP (Instruction Pointer) it forms the address of the next instruction to fetch.
- **DS (Data Segment):** points to the segment containing program data; almost every MOV that references a labeled variable implicitly uses DS.
- **SS (Stack Segment):** points to the segment used for the stack; combined with SP it forms the address of the top of the stack.
- **ES (Extra Segment):** an additional data segment, most often used as the destination segment with string instructions such as MOVSB/STOSB.

Why segmentation exists: the 8086's registers are only 16 bits wide, which alone can address just 64KB, yet its address bus is 20 bits wide (1MB of addressable memory). Segmentation is the trick that reconciles the two: a 20-bit physical address is synthesized from a 16-bit segment value and a 16-bit offset, as you will compute explicitly in Task 5.

Task 3: Pointer and index registers

- **SP (Stack Pointer):** always points to the current top of the stack within the stack segment; it is automatically adjusted by PUSH, POP, CALL, and RET, and you will rely on this heavily in Week 8.
- **BP (Base Pointer):** typically used to reference parameters and local variables that have been pushed onto the stack inside a procedure, since (unlike SP) it stays fixed while you access data at fixed offsets from it.
- **SI (Source Index):** conventionally used as the "read from here" pointer, especially with indexed addressing and string instructions like LODSB/MOVSB.
- **DI (Destination Index):** conventionally used as the "write to here" pointer, especially with STOSB/MOVSB.

Task 4: The Flags register

- **CF (Carry Flag):** set when an arithmetic operation produces a carry/borrow out of the most significant bit; essential for detecting overflow in unsigned arithmetic and for multi-word addition using ADC.
- **ZF (Zero Flag):** set when the result of an operation is zero; this is the flag that JE/JZ and JNE/JNZ actually test.
- **SF (Sign Flag):** set to the sign (MSB) of the result, letting you distinguish a negative result from a positive one after arithmetic or logic instructions.
- **OF (Overflow Flag):** set when a signed arithmetic operation produces a result too large or too small to fit in the destination – the signed counterpart to CF.
- **PF (Parity Flag):** set when the low byte of the result has an even number of 1 bits; historically used for basic error checking on serial communication lines.

Try each of the following in your emulator's debugger and note which flags change: `MOV AL,0FFh` then `ADD AL,1` (watch CF and ZF), and `MOV AL,7Fh` then `ADD AL,1` (watch OF and SF).

Task 5: Physical address calculation

The 8086 forms a 20-bit physical address from a 16-bit segment and a 16-bit offset:

$$\text{Physical Address} = (\text{Segment} \times 10_{16}) + \text{Offset}$$

Example: Segment = 1000h, Offset = 0234h $\Rightarrow$ Physical Address = 10000h + 0234h = 10234h.

Because this formula allows many different segment:offset pairs to resolve to the same physical byte (for example, Segment=1000h/Offset=0244h also equals 10244h... adjust and check this yourself for the address 10234h), the same physical location can legally be reached through more than one "logical address." This is a distinctive and often confusing feature of 8086 segmentation that you should be able to demonstrate with at least two different segment:offset pairs for the same physical address.

Summary:

This week you learned the internal register organization of the 8086, how memory is segmented, and how the Flags register reflects the outcome of

instructions – all of which you will use directly when writing and debugging assembly programs. Keep a printed register/flag reference handy; you will refer back to it constantly for the rest of the semester.

3 Week 3: Introduction to the Assembly Programming Environment

Objective: Set up an 8086 assembly programming environment, understand the structure of an assembly program, and write, assemble, and run a first program.

Tasks:

1. **Task 1:** Install/launch an 8086 assembler and simulator (emu8086, or MASM/TASM under DOSBox), and confirm it can assemble and run a trivial program.
2. **Task 2:** Understand the standard segment structure of an assembly program (.MODEL, .STACK, .DATA, .CODE), and explain the purpose of each directive.
3. **Task 3:** Write a program to display a message on the screen using an INT 21H DOS service, and modify the message text to confirm you understand how the program works end to end.
4. **Task 4:** Assemble, link, and run the program; single-step through it in the debugger and observe how each instruction changes the registers.
5. **Task 5:** Add comments and understand the assemble-link-execute cycle, and be able to explain what happens at each stage if it fails.

Details of Lab Experiment:

Task 2: Structure of an assembly program

Every 8086 assembly program written for MASM/TASM follows a common skeleton: a memory model directive that tells the assembler how large your code/data segments may be, a stack segment that reserves space for procedure calls and local storage, a data segment for declared variables, and a code segment containing the actual instructions. Getting this skeleton memorized is important because a missing **END** label, a missing **ENDP**, or a data segment referenced before **DS** is initialized are the most common beginner errors, and they all trace back to misunderstanding this structure.

Task 3: First program – Hello, World!

```
.MODEL SMALL
.STACK 100H

.DATA
    MSG DB 'Hello, Computer Organization & Assembly!$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AH, 09H          ; DOS function: display string
    LEA DX, MSG           ; load address of string into DX
    INT 21H

    MOV AH, 4CH          ; DOS function: terminate program
    INT 21H
MAIN ENDP
END MAIN
```

Listing 1: Display a message using DOS interrupt 21H

Explanation:

- `.MODEL SMALL` tells the assembler that code and data each fit in one segment (a "small" memory model), which is sufficient for every program in this manual.
- `MOV AX, @DATA` then `MOV DS, AX` initializes the data segment register; note that `DS` cannot be loaded directly from an immediate value on the 8086, which is precisely why this two-step idiom – load into `AX` first, then copy `AX` into `DS` – appears at the top of almost every program you will write.
- DOS function 09H (called via `INT 21H`) prints the `$`-terminated string pointed to by `DX`; forgetting the trailing `$` is a very common bug that causes garbage characters to be printed until a stray `$` byte happens to appear in memory.
- DOS function 4CH terminates the program and returns control to DOS; every program in this manual ends this way, and omitting it can leave the emulator or DOS session in an inconsistent state.

Try it yourself: change the message text, remove the trailing `$`, and re-run the program to see exactly what "garbage output" looks like – this will make the purpose of the terminator immediately obvious.

Task 4: Assemble, link, and run

On a MASM/TASM toolchain, the classic sequence is:

```
MASM hello.asm  
LINK hello.obj  
hello.exe
```

Listing 2: Assemble-link-execute cycle

The **assemble** step (**MASM**) translates your source text into machine code stored in an object file (**.obj**) and reports any syntax errors. The **link** step (**LINK**) resolves segment addresses and produces an executable (**.exe**). The **execute** step actually runs the program under DOS. If assembly fails, the error is almost always a typo, a missing operand, or a mismatched **PROC/ENDP** pair; if linking fails, it is usually a missing **END** label or an undefined external symbol.

In **emu8086**, simply load the source file, click **Compile**, then **Emulate** to step through the program and watch the register/flag panes update instruction by instruction – this single-step view is one of the most valuable debugging tools you have for the rest of the course, since it lets you see exactly what every instruction does to every register and flag in real time.

Summary:

This week you set up your assembly development environment, learned the standard segment structure of an 8086 program, and wrote, assembled, and executed your first program using a DOS interrupt service. Get comfortable with your debugger’s single-step feature now – you will lean on it heavily as programs grow more complex.

4 Week 4: Data Transfer Instructions and Addressing Modes

Objective: Learn to move data between registers, memory, and immediate values, and understand the different addressing modes supported by the 8086.

Tasks:

1. **Task 1:** Use MOV to transfer data between registers, and between a register and memory, and identify which MOV combinations are illegal (e.g. memory-to-memory).
2. **Task 2:** Use XCHG to exchange the contents of two operands, and explain a situation where XCHG avoids the need for a temporary variable.
3. **Task 3:** Demonstrate immediate, register, and direct addressing modes, and explain what data each mode actually operates on.
4. **Task 4:** Demonstrate register-indirect and based/indexed addressing modes, and explain why these are essential for working with arrays.
5. **Task 5:** Use LEA to load an effective address, and explain precisely how it differs from a plain MOV.

Details of Lab Experiment:

Task 1: The MOV instruction

```
.MODEL SMALL
.STACK 100H
.DATA
    NUM1 DW 25
    NUM2 DW ?

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, NUM1      ; load memory into register
    MOV NUM2, AX      ; store register into memory
    MOV BX, 10        ; immediate value into register
    MOV CX, AX        ; register to register

    MOV AH, 4CH
```

```
INT 21H
MAIN ENDP
END MAIN
```

Listing 3: Register and memory data transfer

MOV is the single most frequently used instruction in assembly language, but it has firm restrictions worth memorizing: you cannot move data directly from one memory location to another (you must pass it through a register), you cannot move an immediate value directly into a segment register, and the source and destination operands must be the same size (both bytes or both words).

Task 2: The XCHG instruction

```
MOV AX, 5
MOV BX, 9
XCHG AX, BX      ; now AX = 9, BX = 5
```

Listing 4: Exchanging two register values

XCHG swaps the contents of its two operands atomically in a single instruction, which is exactly what would otherwise require a third temporary register and three separate MOV instructions (MOV TEMP,AX / MOV AX,BX / MOV BX,TEMP). It is a small but genuinely useful instruction that shows up frequently inside sorting algorithms, as you will see in Week 9's bubble sort.

Task 3: Immediate, register, and direct addressing

```
MOV AX, 100      ; Immediate addressing: operand is a
                  literal value
MOV BX, AX       ; Register addressing: operand is inside
                  a register
MOV CX, NUM1     ; Direct addressing: operand is a named
                  memory location
```

Listing 5: Three basic addressing modes

- **Immediate addressing** embeds the operand's actual value directly inside the instruction encoding – it never touches memory or another register to fetch the operand, which makes it the fastest possible way to load a known constant.
- **Register addressing** takes its operand straight from a CPU register – the fastest form of "variable" access since no memory bus transaction is required at all.

- **Direct addressing** references a named memory location by its label; the assembler translates the label into a fixed offset within the data segment at assembly time.

Task 4: Register-indirect and based/indexed addressing

```
LEA BX, NUM1
MOV AX, [BX]           ; Register-indirect: address held in
BX

LEA SI, ARRAY
MOV AX, [SI+2]         ; Indexed: base address plus a
displacement
```

Listing 6: Indirect and indexed addressing

- **Register-indirect addressing** uses a register (BX, SI, or DI on the 8086) as a pointer – the register does not hold the data itself, it holds the *address* of the data, denoted by the square brackets.
- **Based/indexed addressing** adds a constant displacement to a base or index register before dereferencing, which is exactly the mechanism you need to step through consecutive elements of an array (as you will do extensively in Week 9) or to access individual fields of a record laid out in memory.

Without indirect and indexed addressing, every memory access would need a unique, compile-time-fixed label, which would make arrays and loops over data effectively impossible – this is why these modes are considered the conceptual foundation for everything from Week 9 onward.

Task 5: LEA – Load Effective Address

LEA loads the *address* of an operand into a register, unlike MOV, which loads its *value*.

```
LEA DX, MSG           ; DX = offset address of MSG
MOV DX, MSG           ; DX = the word VALUE stored at MSG (
different result!)
```

Listing 7: LEA vs MOV

This distinction trips up nearly every beginner at least once: `LEA DX,MSG` puts the *offset* of the label `MSG` into `DX` (useful for pointing DOS's string-print service at your buffer), whereas `MOV DX,MSG` reads the actual 16-bit *value* stored at that memory location into `DX`. Confusing the two is one of

the most common causes of a program crashing or printing garbage in this course, so always ask yourself "do I want the address, or the value at that address?" before choosing between them.

Summary:

This week you practiced moving and exchanging data between registers and memory, and worked through the immediate, register, direct, register-indirect, and based/indexed addressing modes that the 8086 supports. Every later program in this manual is built entirely out of combinations of these five addressing modes, so revisit this section if a later listing's addressing ever looks unfamiliar.

5 Week 5: Arithmetic Instructions

Objective: Implement basic arithmetic operations in 8086 assembly and understand how the Flags register reflects their results.

Tasks:

1. **Task 1:** Perform addition and subtraction using ADD and SUB, and note which flags each instruction affects.
2. **Task 2:** Use INC and DEC to increment and decrement a value, and explain why they are preferred over ADD/SUB by 1 inside tight loops.
3. **Task 3:** Perform multiplication using MUL and IMUL, and explain where the result is placed for byte versus word operands.
4. **Task 4:** Perform division using DIV and IDIV, and explain what a divide-by-zero does to program execution.
5. **Task 5:** Write a program that adds two multi-digit numbers input by the user, and trace through the ASCII conversion steps by hand before running it.

Details of Lab Experiment:

Task 1: ADD and SUB

```
.MODEL SMALL
.STACK 100H
.DATA
    A DW 45
    B DW 15
    SUM DW ?
    DIFF DW ?

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, A
    ADD AX, B          ; AX = A + B
    MOV SUM, AX

    MOV AX, A
    SUB AX, B          ; AX = A - B
    MOV DIFF, AX
```

```
MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN
```

Listing 8: Basic addition and subtraction

Both ADD and SUB update the Carry, Zero, Sign, Overflow, Auxiliary Carry, and Parity flags according to the result, which is precisely what makes it possible to chain a comparison-style operation (CMP, covered in Week 7) directly after them and branch on the outcome.

Task 2: INC and DEC

```
MOV CX, 5
INC CX      ; CX = 6
DEC CX      ; CX = 5
```

Listing 9: Increment and decrement

INC and DEC add or subtract exactly 1 from an operand. They execute slightly faster and encode more compactly than the equivalent ADD reg,1 / SUB reg,1, and – importantly – they do *not* affect the Carry flag, which matters if you are relying on CF being preserved across a loop counter update while a separate multi-word addition is in progress.

Task 3: Multiplication

```
MOV AL, 6
MOV BL, 7
MUL BL      ; AX = AL * BL = 42, result placed in AX
```

Listing 10: Unsigned multiplication with MUL

MUL performs unsigned multiplication and IMUL performs signed multiplication. Both instructions always use AX (or AL) as one implicit operand and place the product in a fixed location: a byte $\times$ byte multiplication leaves its 16-bit product in AX, while a word $\times$ word multiplication leaves its 32-bit product spread across DX (high word) and AX (low word). Forgetting that the product of two words does *not* fit back into AX alone is a very common source of subtly wrong results.

Task 4: Division

```
MOV AX, 47
MOV BL, 5
```

```
DIV BL          ; AL = quotient (9), AH = remainder (2)
```

Listing 11: Unsigned division with DIV

For a byte divisor, the 16-bit dividend must be in AX, and the result appears as quotient in AL and remainder in AH. For a word divisor, the 32-bit dividend must be spread across DX:AX (so you typically zero DX first with `XOR DX,DX` when dividing an unsigned word), and the result appears as quotient in AX and remainder in DX. On real 8086 hardware, dividing by zero (or a quotient that overflows its destination) raises a CPU exception (INT 0) that halts the program with a runtime error – this is precisely why the calculator project in Week 13 explicitly checks the divisor before calling DIV.

Task 5: Program – Add two numbers entered by the user

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG1 DB 'Enter first digit: $'
    MSG2 DB 0DH,0AH,'Enter second digit: $'
    RESMSG DB 0DH,0AH,'Sum = $'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, MSG1
    MOV AH, 09H
    INT 21H
    MOV AH, 01H          ; read a character
    INT 21H
    SUB AL, 30H          ; convert ASCII digit to numeric
    value
    MOV BL, AL

    LEA DX, MSG2
    MOV AH, 09H
    INT 21H
    MOV AH, 01H
    INT 21H
    SUB AL, 30H
    ADD AL, BL           ; AL = digit1 + digit2

    ADD AL, 30H          ; convert back to ASCII for
    display
    MOV DL, AL
    LEA DX, RESMSG
```

```
MOV AH, 09H
INT 21H
MOV DL, AL
MOV AH, 02H
INT 21H

MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN
```

Listing 12: Reading digits and adding two single-digit numbers

Trace this program by hand for the inputs 3 and 4: the keyboard service returns $AL = 33h$ ('3') and $AL = 34h$ ('4') respectively; subtracting $30h$ from each converts them to the numeric values 3 and 4; adding those numeric values gives 7; and adding $30h$ back converts 7 into the printable character '7' ($37h$). This "subtract $30h$, compute, add $30h$ back" pattern is one you will reuse constantly whenever a program needs to read or print single decimal digits.

Summary:

This week you implemented addition, subtraction, multiplication, and division in assembly and wrote an interactive program that reads two digits from the keyboard and displays their sum. Pay special attention to where operands and results land for MUL/DIV – that placement is fixed by the hardware and cannot be changed.

6 Week 6: Logical and Bit Manipulation Instructions

Objective: Learn the logical and bit-shifting instructions of the 8086 and use them to manipulate individual bits.

Tasks:

1. **Task 1:** Use AND, OR, XOR, and NOT to perform logical operations, and describe a practical use case for each one.
2. **Task 2:** Use TEST to check bit patterns without modifying operands, and explain how it differs from AND.
3. **Task 3:** Use SHL/SHR to perform logical shifts, and relate shifting to fast multiplication/division by powers of two.
4. **Task 4:** Use ROL/ROR to perform bit rotations, and explain how a rotation differs from a shift.
5. **Task 5:** Write a program to check whether a number is even or odd using bitwise logic, and explain why this approach is faster than using DIV.

Details of Lab Experiment:

Task 1: AND, OR, XOR, NOT

```
MOV AL, 0F0H
AND AL, 0FH      ; masks the low nibble
MOV BL, 0F0H
OR BL, 0FH       ; sets the low nibble
MOV CL, 0FFH
XOR CL, 0FH      ; toggles the low nibble
NOT CL           ; inverts every bit of CL
```

Listing 13: Bitwise logical operations

- **AND** is used to *clear* (mask off) specific bits while leaving others unchanged – for example, isolating the least significant bit of a number to test even/odd, as you will do in Task 5.
- **OR** is used to *set* specific bits without disturbing the rest – for example, forcing a particular flag bit in a control register to 1.

- **XOR** is used to *toggle* bits, and also has the famous idiom `XOR AX,AX` which clears `AX` to zero faster and in fewer encoded bytes than `MOV AX,0`.
- **NOT** inverts every bit of its operand; it is a one's-complement operation and is often the first step in manually computing a two's complement, as seen in Week 1.

Task 2: TEST

```
MOV AL, 05H
TEST AL, 01H      ; sets ZF if bit 0 of AL is 0
JZ  EVEN_NUMBER
```

Listing 14: Testing a bit pattern without altering operands

TEST performs the same bitwise AND internally as the **AND** instruction and sets the flags identically, but it discards the result instead of storing it back into the destination operand. This makes it ideal for "peek at a bit, then branch" logic where you need the original operand to remain unmodified for later instructions – exactly the situation in Task 5's even/odd check.

Task 3: Logical shifts

```
MOV AL, 00000011B
SHL AL, 1          ; AL = 00000110B (multiply by 2)
SHR AL, 1          ; AL = 00000011B (divide by 2)
```

Listing 15: SHL and SHR

SHL shifts every bit one position to the left, shifting a 0 into the vacated least-significant bit and shifting the bit that falls off the most-significant end into the Carry flag; shifting left by n positions is mathematically equivalent to multiplying by 2^n . **SHR** shifts every bit one position to the right with the analogous behavior for unsigned division by powers of two. Because a shift executes in a single clock cycle while **MUL**/**DIV** take many cycles, shifting by a constant power of two is a classic performance optimization.

Task 4: Rotations

```
MOV AL, 10000001B
ROL AL, 1          ; bits rotate left, MSB wraps to LSB
ROR AL, 1          ; bits rotate right, LSB wraps to MSB
```

Listing 16: ROL and ROR

The key difference from a shift is that no bits are lost: in a rotation, the bit pushed off one end reappears at the other end instead of being discarded. ROL rotates the pattern left and copies the outgoing MSB into both CF and the new LSB; ROR rotates right and copies the outgoing LSB into both CF and the new MSB. Rotations are frequently used in simple checksum and hashing routines where you want to mix bits around without losing any information.

Task 5: Program – Check even or odd

```
.MODEL SMALL
.STACK 100H
.DATA
    NUM DW 17
    EVENMSG DB 'Number is EVEN$'
    ODDMSG DB 'Number is ODD$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, NUM
    AND AX, 0001H      ; isolate the least significant
                        bit
    CMP AX, 0
    JE EVEN

    LEA DX, ODDMSG
    JMP DISPLAY
EVEN:
    LEA DX, EVENMSG
DISPLAY:
    MOV AH, 09H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 17: Using AND to test the least significant bit

A number is even precisely when its least significant bit is 0, so AND-ing it with 0001H strips away every bit except that one; the result is either 0 (even) or 1 (odd). This bit-trick approach costs a single fast AND instruction, whereas the "obvious" alternative of dividing by 2 with DIV and checking the remainder costs a considerably slower division instruction to obtain exactly the same information.

Summary:

This week you used logical operators and shift/rotate instructions to manipulate bits directly, and applied bitwise AND to determine whether a number is even or odd. These bit-level tricks reappear throughout systems programming (flag registers, packed data formats, simple encryption) well beyond this course.

7 Week 7: Control Transfer Instructions – Jumps and Loops

Objective: Control the flow of an assembly program using unconditional jumps, conditional jumps based on flag values, and the `LOOP` instruction.

Tasks:

1. **Task 1:** Use `JMP` to perform an unconditional jump, and explain why unrestrained use of `JMP` can make a program hard to follow.
2. **Task 2:** Use `CMP` together with conditional jumps (`JE`, `JNE`, `JG`, `JL`), and explain precisely what flags each conditional jump actually tests.
3. **Task 3:** Use `LOOP` to repeat a block of instructions a fixed number of times, and explain which register it relies on and what happens if that register starts at zero.
4. **Task 4:** Implement an `IF-ELSE` construct using conditional jumps, and translate it back into high-level pseudocode to confirm your understanding.
5. **Task 5:** Write a program to find the largest of three numbers, and manually trace it for at least two different input orderings.

Details of Lab Experiment:

Task 1: Unconditional jump

```
JMP SKIP
MOV AX, 1          ; this instruction is skipped
SKIP:
MOV BX, 2
```

Listing 18: `JMP` instruction

`JMP` transfers control to the target label unconditionally, with no flag test involved at all – execution simply continues from the new location. Used sparingly and purposefully (to skip a block, exit a loop early, or implement the “else” branch of a conditional), `JMP` is essential; used carelessly and excessively, it produces the kind of tangled, hard-to-follow control flow that gave rise to the phrase “spaghetti code,” which is exactly why structured constructs like `IF-ELSE` and `LOOP` (Tasks 3 and 4) are built as *disciplined patterns* of jumps rather than arbitrary ones.

Task 2: CMP and conditional jumps

```
MOV AX, 10
CMP AX, 20
JL  LESS           ; jump if AX < 20
JG  GREATER        ; jump if AX > 20
JE  EQUAL          ; jump if AX == 20
```

Listing 19: Comparing two values and branching

CMP internally performs a subtraction (*destination -- source*) purely to update the flags, without altering either operand. Every conditional jump afterward is really just testing a combination of those flags: JE/JZ tests ZF=1; JNE/JNZ tests ZF=0; JG (signed "greater") tests that ZF=0 and SF=OF; JL (signed "less") tests that SF≠OF. Note carefully that assembly language provides *separate* signed (JG/JL) and unsigned (JA/JB) conditional jumps, because the same bit pattern can mean a different comparison result depending on whether you are treating the operands as signed or unsigned – always pick the pair that matches your data's intended interpretation.

Task 3: LOOP instruction

```
.MODEL SMALL
.STACK 100H
.DATA
    SUM DW 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV CX, 10           ; loop 10 times
    MOV AX, 0

AGAIN:
    ADD AX, CX           ; accumulate CX into AX
    LOOP AGAIN           ; decrements CX, jumps to AGAIN
    while CX <> 0
    MOV SUM, AX

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 20: Repeating a block using CX as the counter

LOOP is a compact, dedicated instruction: it decrements CX by 1, and if the new value of CX is not zero, jumps to the target label; otherwise, execution simply falls through to the next instruction. CX is hard-wired as the counter – you cannot substitute another register. A critical trap for beginners: if CX is 0 when LOOP first executes, it wraps around to 0FFFFH instead of stopping, causing the loop to run 65536 times instead of zero times – always double-check that CX is initialized to a genuinely positive count before entering a LOOP-based loop.

Task 4: IF-ELSE using conditional jumps

```
MOV AX, NUM
CMP AX, 0
JGE POSITIVE
    ; ELSE branch
    MOV RESULT, -1
    JMP ENDIF
POSITIVE:
    ; IF branch
    MOV RESULT, 1
ENDIF:
```

Listing 21: IF-ELSE translated to assembly

This is the canonical pattern for translating a high-level `if (condition) {...} else {...}` into assembly: (1) compare and use the *inverse* condition to jump *past* the "then" block into the "else" block, (2) place the "else" code, (3) unconditionally jump over the "then" block to a shared end label, (4) place the "then" code, and (5) mark the shared end label. Every IF, IF-ELSE, WHILE, and FOR construct you write in higher-level languages is compiled down into exactly this kind of compare-and-branch skeleton, so internalizing this pattern here demystifies what a compiler does under the hood.

Task 5: Program – Largest of three numbers

```
.MODEL SMALL
.STACK 100H
.DATA
    A DW 12
    B DW 45
    C DW 30
    MAXVAL DW ?

.CODE
MAIN PROC
    MOV AX, @DATA
```

```
    MOV DS, AX

    MOV AX, A
    CMP AX, B
    JGE CHECK_C_1
    MOV AX, B
CHECK_C_1:
    CMP AX, C
    JGE STORE_MAX
    MOV AX, C
STORE_MAX:
    MOV MAXVAL, AX

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 22: Finding the maximum of three values

The algorithm keeps a running "best so far" value in AX: it first compares A against B and keeps the larger of the two in AX, then compares that running maximum against C and keeps the larger of the two again. Trace this by hand with A=12, B=45, C=30 (AX should end up 45) and again with A=50, B=10, C=20 (AX should end up 50) to convince yourself the two-stage comparison correctly handles every possible ordering of the three inputs.

Summary:

This week you learned to control program flow using unconditional and conditional jumps, implemented loops with `LOOP`, translated an IF-ELSE structure into assembly, and wrote a program to find the maximum of three numbers. These compare-and-branch patterns are the assembly-level foundation of every control structure in every higher-level programming language.

8 Week 8: Procedures and the Stack

Objective: Understand the runtime stack and write modular assembly programs using procedures.

Tasks:

1. **Task 1:** Use `PUSH` and `POP` to manipulate the stack, and explain why the stack grows *downward* in memory on the 8086.
2. **Task 2:** Define a procedure using `PROC/ENDP` and call it with `CALL/RET`, and explain exactly what `CALL` pushes onto the stack.
3. **Task 3:** Pass parameters to a procedure using registers, and discuss a limitation of this approach.
4. **Task 4:** Pass parameters to a procedure using the stack, and explain why `BP` rather than `SP` is used to reference the pushed arguments inside the procedure.
5. **Task 5:** Write a program that computes the factorial of a number using a procedure, and identify the largest input value that will not overflow a 16-bit result.

Details of Lab Experiment:

Task 1: PUSH and POP

```
MOV AX, 100
PUSH AX          ; push AX onto the stack, SP decreases by
                2
MOV BX, 200
PUSH BX
POP CX           ; CX = 200 (last pushed, first popped)
POP DX           ; DX = 100
```

Listing 23: Pushing and popping the stack

The stack is a Last-In-First-Out (LIFO) structure implemented entirely in memory: `PUSH` decrements `SP` by 2 and then stores the 16-bit operand at the new `[SS:SP]`, while `POP` reads the 16-bit value at `[SS:SP]` and then increments `SP` by 2. The 8086 stack grows *downward* (toward lower addresses) by convention, which is why pushing *decreases* `SP` rather than increasing it – a detail that matters the moment you start reasoning about stack-based parameter offsets in Task 4.

Task 2: Defining and calling a procedure

```
.MODEL SMALL
.STACK 100H
.DATA
    RESULT DW ?

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    CALL ADD_NUMBERS
    MOV RESULT, AX

    MOV AH, 4CH
    INT 21H
MAIN ENDP

ADD_NUMBERS PROC
    MOV AX, 15
    ADD AX, 20      ; AX = 35
    RET
ADD_NUMBERS ENDP
END MAIN
```

Listing 24: A simple procedure

CALL does two things atomically: it pushes the address of the instruction immediately following the **CALL** (the "return address") onto the stack, and then jumps to the procedure's entry label. **RET** does the mirror-image operation: it pops that saved return address off the stack and jumps back to it. This is precisely how the processor "remembers" where to resume execution after a procedure finishes – and it is why an unbalanced number of pushes and pops *inside* a procedure (relative to its own **RET**) will corrupt the return address and crash the program.

Task 3: Passing parameters through registers

```
MOV AX, 8
MOV BX, 3
CALL SUBTRACT      ; AX and BX are the "arguments"

SUBTRACT PROC
    SUB AX, BX      ; AX = AX - BX
    RET
SUBTRACT ENDP
```

Listing 25: Register-based parameter passing

Passing arguments in registers is fast and simple to write, which is why it is the natural first technique to learn – but it does not scale well: the 8086 has only a handful of general-purpose registers, so a procedure that needs more than three or four parameters (or that must also preserve the caller's register values) quickly runs out of registers to use, motivating the stack-based convention in Task 4.

Task 4: Passing parameters through the stack

```
PUSH 8
PUSH 3
CALL SUBTRACT_STACK

SUBTRACT_STACK PROC
    PUSH BP
    MOV BP, SP
    MOV AX, [BP+4]    ; first pushed argument
    MOV BX, [BP+6]    ; second pushed argument
    SUB AX, BX
    POP BP
    RET
SUBTRACT_STACK ENDP
```

Listing 26: Stack-based parameter passing

Because `CALL` itself pushes a return address, the arguments pushed by the caller end up *below* that return address on the stack from the procedure's point of view. The procedure therefore saves the caller's BP (`PUSH BP`), then copies the current SP into BP (`MOV BP, SP`) to create a fixed "stack frame" reference point that will not move even as further pushes/pops happen inside the procedure body. From that fixed frame pointer, `[BP+2]` would be the saved BP itself's old value is skipped since we pushed BP after the call – the return address sits at `[BP+2]`, and the arguments sit at `[BP+4]` and `[BP+6]` in this example, because SP (and hence BP) advances by 2 bytes for every word previously pushed. This stack-frame technique is exactly what every high-level language compiler generates to implement function parameters and local variables.

Task 5: Program – Factorial using a procedure

```
.MODEL SMALL
.STACK 100H
.DATA
    N DW 5
    FACT DW ?

.CODE
```



```
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV CX, N
    CALL FACTORIAL
    MOV FACT, AX

    MOV AH, 4CH
    INT 21H
MAIN ENDP

FACTORIAL PROC
    MOV AX, 1           ; running product
FACT_LOOP:
    MUL CX              ; AX = AX * CX
    LOOP FACT_LOOP
    RET
FACTORIAL ENDP
END MAIN
```

Listing 27: Computing 5! using a loop inside a procedure

The procedure keeps a running product in AX, multiplies it by the current value of CX on each pass, and relies on LOOP to both decrement CX and repeat until CX reaches zero – exactly reproducing the mathematical definition $n! = n \times (n - 1) \times \cdots \times 1$. Because AX (and therefore the factorial result) is only 16 bits wide and unsigned MUL of a word by a word can produce a 32-bit product that overflows a single word, this routine only gives a correct 16-bit answer up to $8! = 40320$; $9! = 362880$ already exceeds 65535 and would silently overflow, which is worth confirming by testing $N = 9$ and observing the incorrect (wrapped) result.

Summary:

This week you learned how the stack supports procedure calls, defined and called procedures, practiced two parameter-passing conventions, and wrote a factorial program built around a reusable procedure. Understanding the stack frame built in Task 4 is one of the most transferable skills in this course – it is exactly how function calls work in C, and in nearly every other compiled language, under the hood.

9 Week 9: Arrays and Indexed Addressing

Objective: Declare and manipulate one-dimensional arrays in assembly using indexed addressing.

Tasks:

1. **Task 1:** Declare an array using DB/DW and access individual elements, and calculate the byte offset of an arbitrary element by hand.
2. **Task 2:** Traverse an array using SI/DI and a loop, and explain why the index register must be incremented by 2 (not 1) for a word array.
3. **Task 3:** Write a program to find the sum of array elements, and verify the result by hand for a small test array.
4. **Task 4:** Write a program to find the maximum element of an array, and explain why the algorithm initializes its "running maximum" from the first element rather than from zero.
5. **Task 5:** Write a program to sort an array (bubble sort), and trace at least one full outer pass by hand on paper.

Details of Lab Experiment:

Task 1: Declaring and accessing an array

```
.DATA  
    ARRAY DW 10, 20, 30, 40, 50
```

Listing 28: Array declaration

Individual elements are reached with a base + offset, e.g. the third element (index 2, counting from 0) lies at **ARRAY+4** since each word occupies 2 bytes, so element **i** of a word array always lies at offset **ARRAY + (i * 2)**. This "index times element size" formula generalizes directly to byte arrays (multiply by 1) and to double-word arrays (multiply by 4), and it is precisely what a high-level language's array-indexing syntax like **ARRAY[i]** compiles down to.

Task 2: Traversing an array

```
LEA SI, ARRAY  
MOV CX, 5  
NEXT_ELEM:  
    MOV AX, [SI]
```

```
ADD SI, 2      ; move to the next word
LOOP NEXT_ELEM
```

Listing 29: Walking through an array with SI

SI starts at the array's base address and is advanced by 2 bytes on every iteration because each element is a 16-bit word; if the array were declared with DB instead, SI would need to advance by only 1 byte per element. Getting this step size wrong (e.g. incrementing by 1 for a word array) is a very common bug that causes the program to read the wrong half of adjacent elements instead of whole values.

Task 3: Program – Sum of array elements

```
.MODEL SMALL
.STACK 100H
.DATA
    ARRAY DW 10, 20, 30, 40, 50
    LEN EQU 5
    SUM DW 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, ARRAY
    MOV CX, LEN
    MOV AX, 0
SUM_LOOP:
    ADD AX, [SI]
    ADD SI, 2
    LOOP SUM_LOOP
    MOV SUM, AX

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 30: Summing all elements of an array

Note the use of EQU to define LEN as a compile-time constant equal to the array's element count; this is good practice because it means you only need to update the array declaration and the EQU line (in one place) if the array size ever changes, rather than hunting through every loop in the program for a hard-coded 5. Manually verify: $10 + 20 + 30 + 40 + 50 = 150$, and confirm that SUM holds 0096H (150 in hex) after running the program in your debugger.

Task 4: Program – Maximum element of an array

```
.MODEL SMALL
.STACK 100H
.DATA
    ARRAY DW 23, 67, 12, 89, 45
    LEN EQU 5
    MAXV DW ?

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, ARRAY
    MOV AX, [SI]           ; assume first element is the max
    so far
    MOV CX, LEN-1
    ADD SI, 2
FIND_MAX:
    CMP AX, [SI]
    JGE SKIP_UPDATE
    MOV AX, [SI]
SKIP_UPDATE:
    ADD SI, 2
    LOOP FIND_MAX
    MOV MAXV, AX

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 31: Finding the largest array element

The algorithm seeds AX with the *first* array element (rather than 0) precisely because an array could contain entirely negative numbers – seeding with 0 would then incorrectly “win” against every genuine (negative) element. Because the first element is already accounted for as the seed, the loop only needs to compare the remaining LEN-1 elements, which is why CX is initialized to LEN-1 and SI is advanced past the first element before the comparison loop begins.

Task 5: Program – Bubble sort

```
.MODEL SMALL
.STACK 100H
.DATA
    ARRAY DW 40, 10, 30, 20, 50
```

```

        LEN      EQU 5

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV CX, LEN-1          ; outer pass counter
OUTER:
    PUSH CX
    LEA SI, ARRAY
    MOV CX, LEN-1          ; inner comparison counter
INNER:
    MOV AX, [SI]
    MOV BX, [SI+2]
    CMP AX, BX
    JLE NO_SWAP
    MOV [SI], BX           ; swap adjacent elements
    MOV [SI+2], AX
NO_SWAP:
    ADD SI, 2
    LOOP INNER
    POP CX
    LOOP OUTER

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN

```

Listing 32: Sorting an array in ascending order

Bubble sort repeatedly walks the array comparing each pair of adjacent elements and swapping them if they are out of order, so that on every complete outer pass the largest remaining unsorted element "bubbles up" to its correct final position – which is why the outer loop only needs `LEN-1` passes total. Notice that the inner loop's own counter (`CX`) must be saved with `PUSH CX` before it is reused, and restored with `POP CX` afterward, because `LOOP` for the *outer* pass also depends on `CX` – without this save/restore, the outer loop's counter would be destroyed by the inner loop and the sort would not terminate correctly.

Summary:

This week you declared arrays, traversed them using indexed addressing, and applied that skill to compute a sum, find a maximum, and implement a bubble sort – three of the most common array algorithms. The nested-loop, save-and-restore-`CX` pattern from the bubble sort is broadly reusable any time you need one loop inside another that both rely on the same counter

register.

10 Week 10: String Manipulation Instructions

Objective: Use the dedicated string instructions of the 8086 to copy, compare, and scan blocks of memory efficiently.

Tasks:

1. **Task 1:** Use `MOVSB/MOVSW` with `REP` to copy a string, and explain the role of the Direction Flag (DF).
2. **Task 2:** Use `CMPSB` to compare two strings, and explain the difference between the `REPE` and `REPNE` prefixes.
3. **Task 3:** Use `SCASB` to search for a character within a string, and explain what SI/DI role each string instruction implicitly uses.
4. **Task 4:** Use `LODSB` and `STOSB` to load and store string bytes, and explain when you would use them instead of `MOVSB`.
5. **Task 5:** Write a program to find the length of a null-terminated string, and compare its approach to how the `SCASB` instruction could achieve the same result.

Details of Lab Experiment:

Task 1: Copying a string with `MOVSB` and `REP`

```
.MODEL SMALL
.STACK 100H
.DATA
    SRC DB 'ASSEMBLY LANGUAGE$'
    DEST DB 20 DUP('$')

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV ES, AX

    LEA SI, SRC
    LEA DI, DEST
    MOV CX, 18
    CLD                      ; clear direction flag (auto-
increment SI/DI)
    REP MOVSB                ; repeat MOVSB CX times
```

```
MOV AH, 4CH
INT 21H
MAIN ENDP
END MAIN
```

Listing 33: Copying bytes from one buffer to another

`MOVSB` copies a single byte from `[DS:SI]` to `[ES:DI]` and then automatically advances both `SI` and `DI`, saving you from writing that indexing logic by hand as you did with ordinary `MOV` in Week 9. The Direction Flag (`DF`) controls which way `SI/DI` move after each string operation: `CLD` clears `DF` so `SI/DI` *increment* (process the string left to right), while `STD` sets `DF` so `SI/DI` *decrement* (process the string right to left, useful for reversing or for copying overlapping buffers safely). The `REP` prefix simply repeats the string instruction that follows it `CX` times, decrementing `CX` after each repetition – exactly like an automatic, hardware-accelerated version of the `LOOP`-based copy you would otherwise write by hand.

Task 2: Comparing strings with `CMPSB`

```
LEA SI, STR1
LEA DI, STR2
MOV CX, 10
CLD
REPE CMPSB      ; repeat while equal
JE  STRINGS_MATCH
```

Listing 34: Comparing two byte strings

`CMPSB` compares the byte at `[DS:SI]` with the byte at `[ES:DI]`, updates the flags exactly as `CMP` would, and advances `SI/DI`. `REPE` (equivalently `REPZ`) repeats the comparison while both `CX` is nonzero *and* the bytes compared so far are equal – stopping the instant a mismatch is found or `CX` reaches zero, whichever comes first. Its counterpart, `REPNE` (`REPNZ`), does the opposite: it repeats while bytes are *not* equal, which is the natural choice for a “does this byte appear anywhere in the buffer” search, as used with `SCASB` in Task 3.

Task 3: Searching with `SCASB`

```
LEA DI, TEXT
MOV CX, 20
MOV AL, 'Z'
CLD
REPNE SCASB      ; repeat while not equal
JE  CHAR_FOUND
```

Listing 35: Scanning for a target character

SCASB compares the byte in **AL** against the byte at **[ES:DI]**, updates the flags, and advances **DI** – it is essentially **CMPSB** specialized for “compare memory against a single register value” rather than “compare two memory buffers.” Combined with **REPNE**, this scans forward through up to **CX** bytes of the buffer looking for the first occurrence of the character in **AL**, stopping as soon as a match is found (**ZF=1**) or the buffer is exhausted (**ZF=0**) – exactly the building block behind implementing something like the C standard library’s **strchr**.

Task 4: LODSB and STOSB

```
LEA SI, SOURCE
LODSB          ; AL = [SI], SI is auto-incremented
LEA DI, TARGET
STOSB          ; [DI] = AL, DI is auto-incremented
```

Listing 36: Loading and storing individual string bytes

LODSB loads the byte at **[DS:SI]** into **AL** and advances **SI**; **STOSB** stores **AL** into **[ES:DI]** and advances **DI**. Unlike **MOVSB**, which moves data straight from source to destination in one step, **LODSB/STOSB** route the data *through* **AL**, which is exactly what you need when the byte must be inspected, transformed, or conditionally processed (e.g. converting each character to uppercase) on its way from source to destination rather than copied verbatim.

Task 5: Program – Length of a null-terminated string

```
.MODEL SMALL
.STACK 100H
.DATA
    STR1 DB 'COMPUTER ORGANIZATION', 0
    LENGTH DW 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, STR1
    MOV CX, 0
COUNT_LOOP:
    MOV AL, [SI]
    CMP AL, 0                ; stop at the null terminator
```

```
    JE  DONE
    INC CX
    INC SI
    JMP COUNT_LOOP
DONE:
    MOV LENGTH, CX

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 37: Counting characters until a terminator byte

This routine manually reimplements, byte by byte with an ordinary `MOV/CMP/JMP` loop, exactly the behavior that C's `strlen` performs – and it is very instructive to also rewrite it using `REPNE SCASB` with `AL=0` as an exercise, since that version accomplishes the identical result in a single repeated hardware-assisted instruction instead of an explicit software loop, letting you directly compare the "manual loop" and "string instruction" styles side by side.

Summary:

This week you used the dedicated string instructions – `MOVSB`, `CMPSB`, `SCASB`, `LODSB`, and `STOSB` – together with the `REP` family of prefixes, and applied them to copy, compare, and measure strings. These instructions exist specifically because string processing is common enough that the 8086 designers built dedicated hardware support for it rather than leaving programmers to write everything as generic loops.

11 Week 11: Macros and Modular Programming

Objective: Reduce code repetition and improve readability by defining and using macros.

Tasks:

1. **Task 1:** Understand the difference between a macro and a procedure, and describe a scenario where each is clearly the better choice.
2. **Task 2:** Define and invoke a simple macro using `MACRO/ENDM`, and inspect the assembler's listing file to see how the macro was expanded.
3. **Task 3:** Define a macro that takes parameters, and explain how macro parameters differ from procedure parameters.
4. **Task 4:** Write a macro to display a string on the screen, and use it at least three times in one program.
5. **Task 5:** Write a macro to read a character from the keyboard, and combine it with the display macro from Task 4 in a single short program.

Details of Lab Experiment:

Task 1: Macro vs. procedure

A **macro** is expanded inline by the assembler at *every point* it is invoked – the assembler literally copies and substitutes the macro's body into the surrounding code before generating machine code, so there is no `CALL/RET` overhead at runtime, but the final executable is larger because the same instructions are duplicated everywhere the macro is used. A **procedure**, by contrast, exists exactly once in memory and is invoked at runtime via `CALL/RET`, which keeps the executable smaller but adds a small runtime cost for each call (pushing/popping the return address) and requires you to manage parameter passing explicitly, as you practiced in Week 8. As a rule of thumb: prefer macros for short snippets used frequently where speed matters more than code size (e.g. this week's print-string macro); prefer procedures for longer or recursive logic, or when code size matters more than the small call overhead (e.g. Week 8's factorial routine).

Task 2: A simple macro

```
CLEAR_AX MACRO
    XOR AX, AX
ENDM

; usage:
CLEAR_AX
```

Listing 38: Defining and calling a parameterless macro

The block between `MACRO` and `ENDM` defines a named template; every subsequent line in the source file that simply writes the macro's name (here, `CLEAR_AX`) is replaced, during assembly, by a literal copy of that template's body. If you open the assembler's generated listing file (the `.lst` file MASM/TASM can produce), you will be able to see the expanded instructions inserted exactly where the macro was invoked, which is an excellent way to build intuition for what "macro expansion" really means.

Task 3: A macro with parameters

```
ADD_TWO MACRO OP1, OP2
    MOV AX, OP1
    ADD AX, OP2
ENDM

; usage:
ADD_TWO 5, 10
```

Listing 39: A macro that adds two operands

Macro parameters (`OP1`, `OP2` here) are handled by simple text substitution at assembly time, *not* by passing values through registers or the stack at runtime the way procedure parameters are – the assembler literally replaces the text `OP1` with `5` and `OP2` with `10` everywhere they appear inside the template before generating code. This means macro arguments can be registers, memory operands, or immediate constants completely interchangeably, and different invocations of the same macro can each expand into genuinely different instructions depending on what was substituted in.

Task 4: Macro – Display a string

```
PRINT_STR MACRO MSG
    LEA DX, MSG
    MOV AH, 09H
    INT 21H
ENDM

.MODEL SMALL
```

```
.STACK 100H
.DATA
    GREETING DB 'Assembly Language is fun!$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    PRINT_STR GREETING      ; macro expands to LEA/MOV/INT
    here

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 40: A reusable "print string" macro

This macro packages the three-instruction "load address, set function number, invoke interrupt" idiom you first wrote by hand in Week 3 into a single reusable, self-documenting call. Add two more DB-declared strings to .DATA and print all three with three separate PRINT_STR invocations to see firsthand how much boilerplate a well-designed macro eliminates as a program grows.

Task 5: Macro – Read a character

```
READ_CHAR MACRO
    MOV AH, 01H
    INT 21H
ENDM

; usage:
READ_CHAR      ; character typed by the user ends up in AL
```

Listing 41: A reusable "read character" macro

Combine this with PRINT_STR from Task 4 to build a short interactive program: prompt the user with PRINT_STR, read their response with READ_CHAR, and print an acknowledgment with another PRINT_STR – all using two-line macro invocations instead of repeating six or more raw instructions inline, which is a clear demonstration of how macros improve both readability and maintainability of DOS-service-heavy programs.

Summary:

This week you learned when to prefer a macro over a procedure, defined macros with and without parameters, and built reusable macros for two of the most common DOS operations: printing a string and reading a character. From this point forward, feel free to reuse `PRINT_STR` and `READ_CHAR` in your own programs instead of writing the raw `INT 21H` calls by hand every time.

12 Week 12: Interrupts and I/O Programming

Objective: Understand how software interrupts let a program request services from DOS and the BIOS, and use them for keyboard and screen I/O.

Tasks:

1. **Task 1:** Understand the difference between hardware and software interrupts, and give one real-world example of each.
2. **Task 2:** Use common INT 21H DOS functions for character and string I/O, and explain how the function number in AH determines which service is invoked.
3. **Task 3:** Use INT 10H BIOS video services to set cursor position and clear the screen, and explain why BIOS calls exist alongside DOS calls.
4. **Task 4:** Write a program that reads a line of text and echoes it back, and explain the role of each byte in the input buffer's header.
5. **Task 5:** Write a program that reads a number and displays it in reverse, and extend it (on paper) to handle a two-digit number.

Details of Lab Experiment:

Task 1: Hardware vs. software interrupts

- **Hardware interrupts** are triggered externally by devices (keyboard, timer, disk controller) and occur asynchronously – that is, at essentially any point during a running program's execution, whenever the external device needs attention (e.g. a keystroke has arrived and needs to be buffered).
- **Software interrupts**, such as INT 21H and INT 10H, are triggered explicitly and synchronously by an INT instruction written directly in the program, and are used to call operating system or BIOS routines on demand – essentially a controlled, deliberate "function call" into code that lives outside your own program.

Both mechanisms ultimately work the same way at the hardware level: they look up an entry in the Interrupt Vector Table (a fixed-location table of far pointers at the very start of memory) corresponding to the interrupt

number, and jump to whatever routine is registered there – the only real difference is *what* triggers the jump.

Task 2: Common INT 21H DOS functions

- AH = 01H: read a character from the keyboard (with echo) into AL.
- AH = 02H: display a single character held in DL.
- AH = 09H: display a \$-terminated string whose offset is in DX.
- AH = 0AH: buffered string input into a caller-provided buffer described by DX.
- AH = 4CH: terminate the program and return control to DOS.

INT 21H is a single, general-purpose "system call gate" into DOS – the specific service requested is entirely determined by the function number you place in AH *before* executing INT 21H, exactly the way you have been doing since Week 3's first program. This is conceptually identical to how modern operating systems multiplex many different system calls (open a file, read a socket, allocate memory) through a single system-call instruction, distinguished only by a numeric code placed in a register beforehand.

Task 3: INT 10H video services

```
MOV AH, 02H      ; set cursor position
MOV BH, 0
MOV DH, 5        ; row
MOV DL, 10       ; column
INT 10H

MOV AH, 06H      ; scroll window up (used to clear screen)
MOV AL, 0
MOV BH, 07H
MOV CX, 0000H
MOV DX, 184FH
INT 10H
```

Listing 42: Setting cursor position and clearing the screen

INT 10H is the BIOS's video service gate, distinct from DOS's INT 21H, because it talks directly to the video hardware at a lower level than DOS is concerned with – BIOS calls exist precisely for this kind of hardware-adjacent functionality (video, disk, low-level keyboard access) that predates and sits underneath the DOS operating system layer itself. Function 02H moves the text cursor to a specific row/column (DH/DL) on a given video

page (BH); function 06H, when asked to scroll the entire window (CX=top-left corner, DX=bottom-right corner) up by AL=0 lines, has the side effect of blanking the whole screen – a classic trick for clearing the display in a DOS-era program that predates a dedicated "clear screen" call.

Task 4: Program – Read and echo a line

```
.MODEL SMALL
.STACK 100H
.DATA
    BUFFER DB 50, ?, 50 DUP('$')
    PROMPT DB 'Enter a line of text: $'
    NEWLINE DB 0DH, 0AH, '$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 09H
    INT 21H

    LEA DX, BUFFER
    MOV AH, 0AH           ; buffered keyboard input
    INT 21H

    LEA DX, NEWLINE
    MOV AH, 09H
    INT 21H

    LEA DX, BUFFER+2      ; actual text starts at offset 2
    MOV AH, 09H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 43: Buffered string input using INT 21H function 0AH

Function 0AH expects the buffer it is given to begin with a specific two-byte header: the *first* byte (here, 50) is the maximum number of characters DOS is allowed to read, which you must set before calling; the *second* byte is filled in *by DOS itself* after input completes, holding the actual number of characters the user typed; the actual text (without the header) then begins at offset 2 of the buffer, which is exactly why the program prints starting from BUFFER+2 rather than from BUFFER itself. Forgetting to reserve these

two header bytes, or printing from the wrong offset, is a very common source of confusing off-by-two bugs with this particular DOS function.

Task 5: Program – Display a number in reverse

```
.MODEL SMALL
.STACK 100H
.DATA
    PROMPT DB 'Enter a digit: $'
    NEWLINE DB 0DH, 0AH, 'Reversed: $'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, PROMPT
    MOV AH, 09H
    INT 21H

    MOV AH, 01H
    INT 21H                ; AL holds the ASCII digit typed by
    the user

    LEA DX, NEWLINE
    PUSH AX
    MOV AH, 09H
    INT 21H
    POP AX

    MOV DL, AL              ; a single digit reversed is itself
    MOV AH, 02H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 44: Reversing the digits of a keyed-in number

Note that AX must be saved with `PUSH AX` before the intervening `INT 21H` call that prints the `NEWLINE` label, since that DOS call is free to alter AX internally – this same “preserve a register across a service call by pushing and popping it” pattern is one you will need constantly once you start combining multiple `INT 21H`/`INT 10H` calls in a single routine. As an exercise, extend this program on paper to accept two digits (reusing the ASCII-conversion technique from Week 5’s addition program) and print them back in the opposite order – that generalization is exactly what a

genuine "reverse a multi-digit number" program requires.

Summary:

This week you distinguished hardware from software interrupts, used `INT 21H` and `INT 10H` services for keyboard, string, and screen I/O, and put them together in two interactive programs. Every DOS-based assembly program you write from here on will be built from combinations of exactly these interrupt services.

13 Week 13: Final Project – Simple Calculator and Best Practices

Objective: Integrate the concepts covered throughout the semester – data movement, arithmetic, control flow, procedures, and I/O – into a single menu-driven project, and review best practices for writing clean assembly code.

Tasks:

1. **Task 1:** Design a menu-driven calculator supporting addition, subtraction, multiplication, and division, and sketch its control flow as a flowchart before coding it.
2. **Task 2:** Implement each arithmetic operation as a separate procedure, and explain why this modular structure is easier to test and maintain than one long block of code.
3. **Task 3:** Handle user menu selection using conditional jumps, and explain what happens in this program if the user enters a choice outside 1–4.
4. **Task 4:** Apply best practices: meaningful labels, comments, and modular procedures, and review your own earlier labs for places where these practices were missing.
5. **Task 5:** Test the program with edge cases (e.g. division by zero), and describe, in your own words, why testing edge cases is not optional in real software development.

Details of Lab Experiment:

Task 1–3: A menu-driven calculator

```
.MODEL SMALL
.STACK 100H
.DATA
    MENU DB 0DH,0AH,'1: Add  2: Sub  3: Mul  4: Div',0DH,0
    AH,'Choice: $'
    A DW 20
    B DW 4
    RESULT DW ?

.CODE
MAIN PROC
    MOV AX, @DATA
```

```
MOV DS, AX

LEA DX, MENU
MOV AH, 09H
INT 21H

MOV AH, 01H      ; read the user's menu choice
INT 21H
SUB AL, 30H

CMP AL, 1
JE DO_ADD
CMP AL, 2
JE DO_SUB
CMP AL, 3
JE DO_MUL
CMP AL, 4
JE DO_DIV
JMP DONE

DO_ADD:
CALL ADD_OP
JMP DONE
DO_SUB:
CALL SUB_OP
JMP DONE
DO_MUL:
CALL MUL_OP
JMP DONE
DO_DIV:
CALL DIV_OP
DONE:
MOV RESULT, AX

MOV AH, 4CH
INT 21H
MAIN ENDP

ADD_OP PROC
MOV AX, A
ADD AX, B
RET
ADD_OP ENDP

SUB_OP PROC
MOV AX, A
SUB AX, B
RET
SUB_OP ENDP

MUL_OP PROC
MOV AX, A
```

```
    MOV BX, B
    MUL BX
    RET
MUL_OP ENDP

DIV_OP PROC
    MOV AX, A
    CMP B, 0
    JE DIV_ERROR    ; guard against division by zero
    MOV BX, B
    XOR DX, DX
    DIV BX
    RET
DIV_ERROR:
    MOV AX, 0FFFFH  ; sentinel value signalling an error
    RET
DIV_OP ENDP
END MAIN
```

Listing 45: Simple calculator on two fixed operands, driven by a menu

This program is deliberately structured as a "dispatcher plus four independent procedures" – the main body's only job is to read the menu choice and `CALL` the matching operation, while each operation's own procedure is entirely self-contained and could be tested, debugged, or replaced in isolation without touching the others. Note the fall-through `JMP DONE` when none of the four `CMP/JE` pairs match: with no explicit "invalid choice" handling, an out-of-range entry currently just falls through to `DONE` carrying whatever was last in `AX`, which is a real limitation you should identify in Task 3 and could improve by adding an explicit error message.

Task 4: Best practices in assembly programming

- Use meaningful labels (`ADD_OP`, `DIV_ERROR`) rather than cryptic names like `L1/L2`, since assembly already lacks the self-documenting structure of high-level syntax and relies heavily on naming to stay readable.
- Comment every non-obvious instruction, especially register usage – state *what* a register currently holds and *why*, not just what the instruction mnemonic literally does (the mnemonic is already visible; the intent is what needs explaining).
- Break the program into small, single-purpose procedures, exactly as this calculator does, so each piece can be understood, tested, and reused independently.
- Preserve registers you did not intend to change (`PUSH/POP` around a procedure body when needed), so that calling your procedure never surprises the caller by silently clobbering a register it was still using.

- Guard against edge cases such as division by zero, as shown in `DIV_OP` above, rather than assuming input will always be well-behaved.

As a concrete exercise, go back to two or three of your earlier weekly labs and identify at least one place where a label could be renamed more descriptively or a comment could be added to clarify intent – this kind of retrospective review is exactly how professional codebases stay maintainable over time.

Task 5: Testing edge cases

Re-run the calculator with `B` set to 0 and choice 4 (division) to confirm the `DIV_ERROR` path is taken instead of the processor raising a divide-by-zero exception that would otherwise halt the program abruptly. Also try choice 5 (an out-of-range menu option) and observe the fall-through behavior discussed in Task 1–3 above. Deliberately testing boundary and invalid inputs like these – not just the “happy path” where everything goes as expected – is precisely what separates a program that merely *seems* to work from one that is genuinely reliable, and it is a habit worth carrying forward into every programming course and job that follows this one.

Summary:

In this final week you combined data movement, arithmetic, conditional branching, and procedures into a single menu-driven calculator, and reviewed the coding practices that keep assembly programs readable, modular, and robust. Congratulations on completing every core building block of 8086 assembly language – number representation, registers and addressing, arithmetic and logic, control flow, procedures and the stack, arrays, strings, macros, and interrupts – that this course set out to teach.